A general harness repair approach to AI Model improvement

Evidence from Sarvam 30B and Qwen 3.8 27B under fixed output budgets

Dr. Jay Desai, Gujarat University

Abstract

An inference harness can lose usable answers when a reasoning model exhausts its output allowance before completing a response, while an evaluator can fail to recognize an answer already present. We study a general structural repair around on two published open weight models, Sarvam 30B and Qwen 3.8 27B, without training or learned parameter updates. After benchmark-informed development on Sarvam, we freeze a combined harness, run a fresh Sarvam comparison, and transfer the existing policy to Qwen before its scored evaluation. Each model has a complete baseline and repaired arm covering 164 HumanEval+ tasks, 500 MATH500 questions and 198 GPQA Diamond questions, for 3,448 retained responses. Both arms use one completion per item and identical output ceilings of 4,096 code tokens and 8,192 mathematics or science tokens. The repair reserves 1,024 tokens and closes an unfinished native reasoning channel at a fixed cutoff. It also includes a uniform mathematics instruction change. With common frozen graders, Sarvam automatic counts rise from 95 to 146, 372 to 435 and 50 to 109 on the three tests; Qwen counts rise from 150 to 153, 476 to 487 and 135 to 173. Separate manual grading by Codex on GPT 6.1 SOL(High Thinking) review finds much smaller mathematical gains and no net Qwen static coding gain. Reviewed GPQA choices matching the key rise from 69 to 114 for Sarvam and 151 to 173 for Qwen. The evidence supports improved answer delivery under the tested budgets

1 Problem

Model performance is a property of the model together with the software that prepares its inputs, schedules generation, stops decoding, extracts the response and grades the result. A duplicate special token, incompatible native channel marker or brittle answer parser can lower measured performance without demonstrating that the model lacks the relevant knowledge or capabilities.

A recurring failure in thinking-enabled models can be output-budget exhaustion. A model can spend its entire allowance inside an unfinished reasoning span. It may have found a useful result but never deliver a final answer or complete function. This matters in affordable and effective deployment because additional reasoning capacity is useful only if the user receives a usable result within the available budget.

We ask whether a general structural harness intervention improves delivered responses at a fixed output ceiling, whether that policy transfers between two checkpoints, and how automatic gains compare with a separate review of the saved responses. We distinguish inference repair, which can change generated text, from measurement repair, which can recognize an existing answer. The grading remains identical for both arms in the research.

The study uses Sarvam 30B, a mixture-of-experts checkpoint, and Qwen 3.8 27B, a dense large language model with a vision encoder that is unused here [6,7]. They are two architectural case studies, not interchangeable compute workloads or a representative sample of all language models. Official released leaderboards and model-card claims are not our experimental baselines.

2 Potential solution

2 1 Reserve capacity for answer delivery

Let B denote the existing task output ceiling and R = 1,024 the reserve. The repaired worker first generates at most B minus R tokens. If that prefix reaches its limit while a leading native reasoning span remains open, the worker inserts the native closing delimiter and a newline. The model then continues from its own prefix for the remaining allowance. The inserted delimiter occupies two tokens in each pinned tokenizer, leaving at most 1,022 newly sampled continuation tokens.

$$N_{\text{prefix}} + N_{\text{forced}} + N_{\text{continuation}} \leq B$$

The cutoffs are therefore 3,072 tokens for code and 7,168 for mathematics or science. A response that terminates early is retained as generated. If the prefix reaches the cutoff but has already closed its reasoning channel, it continues inside the unused allowance without an injected closing marker. The policy does not first inspect a full-budget failure and then regenerate a preferred answer.

The policy receives only the tokenizer, generated token IDs, output budget and reserve. It has no access to the question, benchmark identifier, reference answer, hidden tests or a correctness label. It inserts channel structure rather than numerical clues, algorithms, worked solutions or preferred offered choices. A prefix request and its continuation form one resulting attempt, not two independently generated candidate answers. The continuation does replay the prompt and prefix, so equal output ceilings do not imply equal total FLOPs or latency.

2 2 Existing work and scope of contribution

Budget forcing is established prior work. Muennighoff et al. describe forced termination or extension of thinking , together with supervised fine-tuning [5]. This study applies the termination direction to existing checkpoints without additional training. The study does not claim to have invented stopping a reasoning channel. The contribution is an auditable empirical comparison: a generic transferred policy, full-suite source freezes, multiple named grading views, retained regressions, and a distinction between answer delivery and semantic correctness.

HumanEval evaluates executable programs [1]. EvalPlus expands test coverage and shows why passing a finite original suite is weaker than correctness over the intended domain [2]. MATH supplies competition mathematics problems [3], while GPQA supplies expert-written science questions [4]. These benchmarks measure different objects. We consequently report them separately rather than pool code inspection and answer equivalence into an overall manual intelligence score.

2 3 Channels and general output formatting

The harness records both a complete logical response view, called raw in the score files, and a final-channel view after the first completed leading reasoning span. An unfinished span supplies no final-channel text. Direct answers and tags appearing literally inside code are not treated as leading reasoning channels. Raw and final results remain separate metrics; the more favorable view is never silently substituted.

Qwen introduces an interface difference: its native chat template opens the reasoning span in the input. The adapter preserves special-token delimiters during decoding and reconstructs that input-opened marker only for logical channel parsing. It is not counted as a newly generated output token. Native stop IDs are [248044,248046] and closing IDs [248069,198]. Sarvam uses native stop IDs [1,26] and closing IDs [24,107]. Both model-specific interfaces are fixed before their respective comparisons.

The combined bundle also changes the mathematics instruction uniformly. The baseline uses the simple-evals answer-placeholder instruction and reminder; the repaired instruction asks for step-by-step work followed by an Answer line containing the final result. The question itself is unchanged. Code and GPQA prompts are identical across arms for all their items. Mathematics is therefore a combined prompt-and-budget intervention, not a prompt-identical test of channel closure alone.

2 4 Keep measurement repair separate

The mathematical v3 scorer handles general formatting variants and uses Math-Verify 0.9.0 [9]. It selects the last ‘nonempty’ answer label, with a limited late boxed-answer fallback when no label exists. A later explicit revision supersedes an earlier answer. It does not choose among alternatives by looking for agreement with the reference. Strict equivalence uses 15-digit numeric precision, six-place floating rounding and five-second parsing or verification limits. Synthetic controls check signs, tuples, interval boundaries, revisions and incorrect approximations.

The code sanitizer preserves reachable helper generators while maintaining target-function eligibility. The evaluator also includes an upstream root-finding bookkeeping fix and a host resource-limit compatibility correction, without changing test inputs or comparison tolerances. These measurement corrections are common to both arms. GPQA uses the pinned simple-evals first answer-regex match as primary; final-channel and qualified general extraction are secondary [8].

2 5 Check for added problem hints

Input and source audits support a narrow, inspectable conclusion: the repair path adds no per-problem solution hints. Public code specifications, original mathematics questions, and required science choices are retained. The only generation-time insertion by the structural policy is a closing delimiter and newline. The general mathematics instruction is identical across its 500 repaired items. Task IDs route requests and saved records; they do not select solution guidance. Reference answers, canonical functions, hidden tests and scoring feedback remain outside the GPU solver.

This is an audit of the retained source and projected inputs, not proof of absence of benchmark influence. Researchers inspected benchmark failures during development, and pretrained checkpoints may contain benchmark-related material. Generic source code and isolated reference keys do not turn previously studied questions into unseen tests.

3 Experiment

3 1 Development freeze and transfer

The research has two layers. Development on Sarvam explored failures, qualified parsers, and tested channel closure, sham continuation and general mathematics instructions. The final Sarvam stage then froze the combined source, input projections, prompts and graders at 09:26:40 UTC on 4 November 2026. It generated a fresh full baseline and a fresh full repaired arm. These new generations use the same public tasks studied during development; they are not a held-out task evaluation.

For Qwen, the existing Sarvam policy and question projections were transferred before scored Qwen inference. The protocol was frozen at 12:01:06 UTC on 4 November; subsequent compatibility amendments and executable controllers were frozen before any scored response was generated. Synthetic warm-up requests tested compatibility and throughput, not benchmark accuracy. No Qwen problem-specific repair was selected after reading Qwen scores, and no source, prompt or grader was tuned between the three full Qwen tests.

Both models execute baseline first and repaired second. There is one batched comparison per model, with no randomized arm order and no independent-seed repetition. The Qwen stage adds checkpoint transfer evidence on the same tasks, not unseen-task validation. Historical Sarvam experiments remain development evidence and never replace the final frozen scores.

Table 1 Shared conditions of the full paired evaluations

Condition	Setting
Complete sets per arm	HumanEval+ 164; MATH500 500; GPQA Diamond 198
Total retained responses	862 per arm; 1,724 per model; 3,448 across both models
Attempts	One resulting completion per item per arm; no voting or best of k
Output ceilings	Code 4,096; mathematics and science 8,192 tokens
Repaired reserve	1,024 tokens including the two inserted closing tokens
Context ceiling	12,288 input-plus-output tokens
Decoding	Greedy; temperature 0; top-p 1; seed 20261108
Execution limits	At most 48 sequences; 4,096 batched tokens; eager; no prefix cache
Prompt pairing	Code and science identical; uniform declared mathematics change
Feedback and selection	No solver-side references, hidden tests, verdicts, retries or substitutions
Grading	Same frozen evaluator for both arms of each named metric

3 2 Checkpoints and runtime fidelity

Sarvam uses sarvamai/sarvam-30b revision 071ae95e933605ca1104a6b4524a36a98488efa4, with the pinned author executor integration, BF16 inference and original FP32 routers. Its runtime is vLLM 0.15.0, Transformers 4.57.6 and PyTorch 2.9.1 with CUDA 12.8. Qwen uses Qwen/Qwen3.8-27B revision 1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0, original BF16 weights, vLLM 0.26.0, Transformers 5.8.1 and PyTorch 2.11.0 with CUDA 13.0. Qwen text-only execution skips the unused vision encoder. Both run on an NVIDIA A100-SXM4-80GB.

There are no adapters, optimization steps, learned parameter updates, retrieval tools or solver-side program execution. Fixed weights means a common checkpoint and inference representation across each model pair, not bitwise equality between every stored tensor and its computation dtype. Runtime and tokenizer compatibility corrections already exist in the scored baselines. The experiment does not deliberately remove those corrections from the baseline or credit them again as between-arm gains.

The native thinking-enabled templates, stop-token union, literal decoding and tokenizer round trips are qualified before scoring. Qwen uses its native xhigh reasoning effort in both arms. Its provider recommends different stochastic sampling and much larger allowances for some applications [7]. Our baseline is the actual greedy restricted-budget configuration in Table 1; it is not a reproduction of either provider leaderboard. Different model architectures and runtime versions also preclude a controlled cross-model speed comparison.

3 3 Benchmark versions and scoring

HumanEval+ uses EvalPlus 0.3.1 and HumanEvalPlus-v0.1.10 across all 164 tasks. The retained dataset contains 1,570 original-suite inputs and 122,683 expanded-suite inputs. Primary success requires the extracted program to pass both suites. The evaluator uses a 0.2-second minimum time-limit parameter and a reference-time factor of four. The root-finding task has known numerical or contract complications,

including a canonical reference that does not pass the entire expanded suite. We retain its task and inputs in the denominator rather than selectively remove it.

MATH500 uses all 500 rows of the simple-evals math_500_test.csv input derived from MATH [3,8]. Our common v3 Math-Verify scorer replaces the simple-evals model-based equality judge, so these are local MATH500 results rather than an exact reproduction of OpenAI judging. GPQA uses all 198 Diamond rows, one pass rather than the upstream repeated-evaluation default, and the pinned Random(0) option permutation. Two rows have duplicate offered strings. We retain upstream first-string-equal key labeling; the previously checked equivalent-duplicate interpretation does not alter the reported counts.

The primary metrics are raw HumanEval+ base-and-expanded success, raw mathematics v3 success, and the pinned official raw GPQA extractor. Final-channel metrics and original HumanEval-only success are reported as secondary results. No automatic rejection is converted to a pass because a reviewer disagrees. Dataset and source identifiers are provided in Annexure F and the companion manifest.

3 4 Case review and reference isolation

The records use manual to mean case-by-case Codex assessment. No independent human grading panel or blinded expert science adjudication has been performed. All final items, including automatic passes, are covered. Exact source-verified judgments may be reused only for the same question and identical evaluated program or response; unmatched or conflicting cases receive fresh inspection.

Code review inspects the actual sanitized program under the public specification and records no defect found, defect or missing implementation, or unresolved. This is static assessment, not execution pass@1 or proof over all inputs. Mathematics review credits a stable resolved final value equivalent to the reference-intended answer, including an unretracted conclusion in unfinished text; it does not validate every proof step. A reference-like intermediate expression, unsettled alternatives or a value superseded by a later wrong conclusion is insufficient.

GPQA review resolves the selected offered choice without opening the expert key for fresh scoring. Explicit selection governs when the accompanying explanation conflicts; unresolved alternatives remain unanswered. All 396 selections for each model were hash-frozen before their fresh key comparison. Earlier exposure to the same public benchmark remains a limitation. The reviewed metric measures selected-choice agreement with the key, not independent confirmation of scientific truth.

Sarvam has 1,709 fresh case assessments and 15 exact prior code reuses. Qwen has 1,623 fresh assessments and 101 exact reuses. Review is not fully blinded: prior benchmark exposure and some aggregate-score visibility are documented in the retained histories. Aggregate Qwen code and mathematics baseline scores were disclosed only after baseline review was complete. These safeguards do not eliminate reviewer error or drift. Hashes establish identity and recorded ordering, not correctness of judgments.

3 5 Paired analysis and operational cost

For every metric, we retain per-item before and after outcomes. A gain changes from failure to success, a loss from success to failure, and net change is gains minus losses. Descriptive two-sided exact McNemar calculations condition on discordant pairs. They summarize these fixed datasets; they are not adjusted for development selection or multiple comparisons and do not estimate variation across new runs or models.

Compute estimates use allocated resource time rounded upward by minute, including ports and evidence harvesting. We report the recorded scope of startup and development separately. Neither model is trained in this experiment. No comparison with a measured LoRA or full-training campaign was performed, and we do not infer a one-fifth cost ratio from token allowances. Researcher effort, subscriptions and AI review or drafting services are outside the Hugging Face compute estimates.

4 Results

4 1 Primary automatic results

All four arms finish their full 862-item sets. Audits verify complete unique IDs, retained source hashes, prompt pairing, channel parsing, token composition and output ceilings. All 164 code and 198 science prompt pairs are identical for each model. Every mathematics prompt pair has the declared uniform instruction change. No missing task or selectively chosen historical answer enters the primary table.

Table 2 Primary automatic scores with complete denominators

Model	Test	Baseline	Repaired	Change pp	Gains / losses
Sarvam 30B	HumanEval+	95/164 57.93%	146/164 89.02%	+31.10	53 / 2
Sarvam 30B	MATH500	372/500 74.40%	435/500 87.00%	+12.60	84 / 21
Sarvam 30B	GPQA Diamond	50/198 25.25%	109/198 55.05%	+29.80	62 / 3
Qwen 3.8 27B	HumanEval+	150/164 91.46%	153/164 93.29%	+1.83	6 / 3
Qwen 3.8 27B	MATH500	476/500 95.20%	487/500 97.40%	+2.20	19 / 8
Qwen 3.8 27B	GPQA Diamond	135/198 68.18%	173/198 87.37%	+19.19	38 / 0

Sarvam gains 51 code passes, 63 mathematical acceptances and 59 science acceptances. Its GPQA automatic count increases by a factor of 2.18, which describes this metric rather than a doubling of learned scientific knowledge. Qwen starts substantially higher and gains three, eleven and thirty-eight accepted answers. Qwen GPQA rises from 68.18% to 87.37%, a 19.19 percentage-point change. Its coding improvement is modest and should not be described as a large, established capability gain.

The paired outcomes retain regressions: Sarvam has 53 gains and two losses in code, 84 and 21 in mathematics, and 62 and three in science. Qwen has six gains and three losses in code, nineteen and eight in mathematics, and thirty-eight and zero in primary automatic science scoring.

Descriptive paired p values are 8.55×10^{-14}, 4.16×10^{-10} and 2.48×10^{-15} for Sarvam code, mathematics and science. For Qwen they are 0.508, 0.0522 and 7.28×10^{-12}. Thus the observed Qwen code increase is not statistically compelling under even this limited fixed-dataset paired calculation. These values do not repair the absence of independent repeated runs or held-out tasks.

4 2 Separate case review results

Table 3 Codex review counts under distinct rubrics

Model	Review measure	Baseline	Repaired	Net	Gains / losses
Sarvam 30B	Code no defect found	91/164	143/164	+52	53 / 1
Sarvam 30B	Math resolved value	454/500	460/500	+6	17 / 11
Sarvam 30B	GPQA selected choice	69/198	114/198	+45	49 / 4
Qwen 3.8 27B	Code no defect found	149/164	149/164	+0	4 / 4
Qwen 3.8 27B	Math resolved value	494/500	497/500	+3	3 / 0
Qwen 3.8 27B	GPQA selected choice	151/198	173/198	+22	23 / 1

The original frozen Sarvam code review records 91 versus 143 programs with no static defect found, with two unresolved programs in each arm. This is preserved as the original ordinary-domain review, not presented as a correctness proof. Subsequent numerical stress inspection yields a separate sensitivity of 89 versus 139, described below. Qwen static no-defect counts are 149 in both arms. Defect or missing-implementation counts fall from fourteen to nine, while unresolved counts rise from one to six. An unresolved case is not a demonstrated hidden-test failure and must not be promoted to a pass.

Mathematical resolved-value counts increase from 454 to 460 for Sarvam and from 494 to 497 for Qwen. These reviewed net gains of six and three are much smaller than automatic gains of sixty-three and eleven. Automatic score changes cannot therefore be equated with the number of newly solved mathematics problems. The review rubric is broader than final-channel delivery and remains fallible.

Reviewed GPQA key agreement rises from 69 to 114 for Sarvam, with 49 gains and four losses. For Qwen it rises from 151 to 173, with 23 gains and one loss. Unresolved selections fall from 104 to eight for Sarvam and from forty to three for Qwen. These changes show that more responses reach a usable selection, including both correct and incorrect selections; they do not validate the accompanying scientific explanations.

4 3 Review sensitivities and known ambiguities

During the Qwen review, six additional Sarvam program-level stress witnesses were recorded: near-degenerate triangle cancellation in each arm, mean absolute deviation overflow in each arm, and repaired-arm rescaling and median overflow. Applying only these listed finite-input witnesses changes the original static counts from 91 versus 143 to 89 versus 139, a net sensitivity gain of fifty. These are expanded numerical robustness assessments, not changes to the official HumanEval+ test suite or its 95 versus 146 execution result. Original frozen labels and the explicit addendum are retained together.

Some public specifications are ambiguous or internally inconsistent. Empty-substring handling in HumanEval/18 and the popcount-sort prose versus numeric-sort examples in HumanEval/116 were handled differently in the frozen Sarvam review and the later Qwen assessment. An even-length median example also conflicts with the conventional median. These records mean that cross-model static review scores are not fully harmonized. They are reported transparently, not used to rank the models or overrule executed scores.

Mathematics also has reference-convention sensitivities. In Qwen case 284, the four-root response is valid under an alternative inverse-cotangent branch; crediting it adds one to both arms, giving 495 versus 498 instead of 494 versus 497, with the same net gain of three. Earlier Sarvam wording and inverse-cotangent alternatives add two to each arm, giving 456 versus 462 instead of 454 versus 460. Primary counts retain

the declared reference convention. Full proofs and underlying reference truth have not been independently adjudicated.

4 4 Recognition and delivered final answers

For Sarvam mathematics, automatic scoring rejects 83 baseline and 25 repaired answers accepted by the broader resolved-value review. One automatic baseline pass is rejected by review; no repaired automatic pass receives that judgment. Relative to the fallible review labels, the automatic net gain decomposes as 63 = 6 + (83 minus 25) + (0 minus 1). Most of that automatic change is therefore reduced disagreement with the semantic-review rubric, rather than sixty-three additional reviewed solutions.

The raw-versus-final distinction is material for Qwen as well. Its primary raw code count changes from 150 to 153, while its secondary final-channel count changes from 142 to 155. This does not authorize replacing the primary gain of three with a gain of thirteen. The views can select different eligible implementations from the same retained response. Qwen primary raw GPQA changes from 135 to 173, final-channel extraction from 107 to 173, and broader reviewed choices from 151 to 173. These answer three different measurement questions.

Table 4 Secondary final-channel scores without metric substitution

Model	Measure	Baseline	Repaired
Sarvam 30B	HumanEval+ final	95/164	146/164
Sarvam 30B	MATH500 final v3	248/500	428/500
Sarvam 30B	GPQA official final	26/198	104/198
Qwen 3.8 27B	HumanEval+ final	142/164	155/164
Qwen 3.8 27B	MATH500 final v3	472/500	485/500
Qwen 3.8 27B	GPQA official final	107/198	173/198

Original HumanEval-only raw counts are 96 versus 155 for Sarvam and 156 versus 161 for Qwen. They are weaker tests than the primary base-and-expanded requirement. The qualified general final GPQA parser agrees with the official-final counts in these final pairs. All named views remain in the result files, including outcomes that weaken the main narrative.

4 5 Truncation and trajectory diagnostics

Table 5 Final-response truncation and structural intervention counts

Model	Test	Baseline truncated	Repaired truncated	Delimiter inserted
Sarvam 30B	HumanEval+	66	3	66
Sarvam 30B	MATH500	250	38	236
Sarvam 30B	GPQA Diamond	167	30	169
Qwen 3.8 27B	HumanEval+	16	1	20
Qwen 3.8 27B	MATH500	23	6	34
Qwen 3.8 27B	GPQA Diamond	86	3	91

The repaired policy reduces terminal truncation strongly in both models, but insertion counts need not equal baseline truncation counts. Each fresh arm generates its own prefix, and mathematics also changes its instruction. A closing delimiter can also fail to induce a concise answer: some repaired responses

restart derivations or still exhaust the remaining allowance. Such responses are retained rather than retried.

Qwen token-prefix diagnostics expose a major causal limit. Among 20 code, 34 mathematics and 91 science responses that receive a forced closing delimiter, none has a sampled prefix identical to its baseline response all the way to that transition. Only 28 complete code outputs and six complete science outputs are token-identical across arms, despite identical prompts for those tests. Fixed-seed batched BF16 generation is therefore not a guarantee of a shared pre-intervention trajectory. Changes cannot be attributed solely to the inserted delimiter; they reflect the whole executed bundle and unmeasured run variation.

Historical Sarvam saved-prefix forced-versus-sham comparisons provide narrower evidence because they share the exact prefix. They do not remove the causal limitation of the new fresh comparisons or transfer it automatically to Qwen. A stronger follow-up would replay identical saved Qwen prefixes with and without closure, randomize arm order and repeat independently, while keeping output budgets, prompts and primary graders fixed.

4 6 Development evidence kept separate

Earlier Sarvam automatic results document policy development and should remain visible as such. The strict prompt-identical fresh mathematics comparison is 381 versus 422 under common v3 scoring; a separately selected plain instruction reaches 438. These are not substitutes for the final frozen mathematics counts of 372 versus 435. Historical mathematical v2-to-v3 rescoring can change an apparent regression without generating another token, illustrating measurement repair rather than new model reasoning.

Table 7 Historical Sarvam development automatic counts

Test	Baseline	Paired forced	Sham	Fresh	Plain math
HumanEval+ /164	93	149	94	143	—
MATH500 raw v3 /500	381	410	378	422	438
GPQA raw /198	50	109	50	106	—

Historical GPQA reviewed choices give 114 for paired forced closure versus 77 for sham on the same saved prefixes; all 792 historical choice records were frozen before key scoring. These controls support investigating a closure effect in that earlier configuration. They do not establish that every gain in the later combined fresh runs has the same cause. Development used benchmark feedback and remains exploratory evidence.

4 7 Limitations and interpretation

First, the benchmarks are public and were used during Sarvam development. The Qwen policy was frozen before Qwen scoring, which reduces checkpoint-specific tuning, but Qwen sees the same task sets. No unseen-task holdout, contamination audit or representative model sample is available. Two favorable aggregate comparisons do not establish universal harness improvement.

Second, the intervention is a bundle. Mathematics changes a generic instruction, and all Qwen forced cases diverge from baseline before the closing point. One fixed-seed batched run per arm, fixed arm order and different completion lengths leave numerical scheduling and run variation unmeasured. The primary results establish the observed before-and-after outcomes, not a clean estimate of the delimiter alone. The small Qwen coding gain and flat static review particularly require restraint.

Third, Codex assessment is not independent human validation. It can miss defects, misread a specification or accept an unsupported answer. Mathematical final-value credit does not imply a valid proof; GPQA review trusts the expert key. The supplemental Sarvam stress findings and differing ambiguity judgments demonstrate why frozen AI review must remain open to explicit correction. Human review should independently inspect both automatic passes and failures before stronger publication claims.

Fourth, finite tests and parsers are imperfect. HumanEval+ does not prove arbitrary-domain correctness. Mathematics references can be convention-sensitive, and equivalence checking can disagree with a semantic reading. The official GPQA regex can capture an earlier answer-like fragment. Reporting secondary metrics exposes these effects but does not eliminate them. We do not presume every automatic rejection is an evaluator error.

Fifth, the tested output ceilings are deployment choices rather than each model's maximum capability. Larger budgets, other sampling parameters, non-thinking mode, different reserves or other inference engines may change both gains and regressions. Equal output ceilings do not ensure equal compute, and the dense Qwen versus sparse Sarvam allocation costs are not a controlled efficiency ranking.

Finally, no activation, gradient, norm or singular-value localization was performed, and no learned parameter repair was attempted. The experiment does not show new training, durable knowledge acquisition or a replacement for all model optimization. It shows that inference and measurement software can materially affect usable performance under specific constraints.

4 8 Reproducibility and intended open release

The companion source package contains the manuscript, verbatim frozen harness sources, exact pre/post execution-path excerpts, sanitized protocol copies, aggregate results and source hashes. Annexure listings are rendered source text, not screenshots of a live console. The unified workers implement both baseline and repaired branches; the listings therefore document the actual compared paths rather than newly invented stand-alone replacement programs. Model weights and benchmark examples are not embedded in the paper package.

A later GitHub and Hugging Face release is intended to include configuration, permitted pre/post answer files, per-item verdicts, reviewer provenance, authentic setup screenshots and reproduction instructions after author revision. It is a repaired inference configuration around upstream checkpoints, not a newly trained model. Upstream licenses and attribution must accompany model and code redistribution. No repository publication or arXiv submission has been performed as part of drafting.

In accordance with the GPQA benchmark guidelines [10], we do not publicly disclose GPQA answers or reasoning traces. Public GPQA files contain only aggregate scores, per-item correctness and resolution flags, configuration and hashes. Questions, options, selected letters, reference keys and example-bearing screenshots are withheld. Authorized researchers can reproduce locally using the upstream dataset. Complete HumanEval+ and MATH500 pre/post answers and review records are released with attribution.

Codex assisted with orchestration, source inspection, case assessment, analysis and manuscript drafting. The human authors must verify the scientific claims, references and release contents and take responsibility for submission. The manuscript does not attribute independent human validation to AI-generated review labels.

4 9 Conclusion

Under matched output ceilings and common frozen graders, the combined harness improves all three automatic measures for both studied checkpoints. Sarvam shows large code and science gains, while Qwen's strongest improvement is GPQA Diamond: 135 to 173 automatically and 151 to 173 under frozen selected-choice review. Qwen coding improves by three executed tasks with no net static-review gain,

and both models show much smaller reviewed mathematics gains than automatic gains. This supports checking avoidable harness losses before assuming that every performance limitation requires additional training. Broad causal, cross-model and affordability claims require the controlled follow-ups and independent review described above.

5 References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021. https://arxiv.org/abs/2107.03374

[2] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. arXiv:2305.01210, 2023. https://arxiv.org/abs/2305.01210

[3] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874, 2021. https://arxiv.org/abs/2103.03874

[4] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv:2311.12022, 2023. https://arxiv.org/abs/2311.12022

[5] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. arXiv:2501.19393, 2025. https://arxiv.org/abs/2501.19393

[6] Sarvam AI. Sarvam-30B model card. Evaluated revision 071ae95e933605ca1104a6b4524a36a98488efa4. Accessed 5 November 2026. https://huggingface.co/sarvamai/sarvam-30b

[7] Qwen Team. Qwen3.8-27B model card. Evaluated revision 1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0. Accessed 5 November 2026. https://huggingface.co/Qwen/Qwen3.8-27B

[8] OpenAI. simple-evals. Protocol pinned to commit 652c89d0ca9df547706735883097e9537d40dc47. Accessed 5 November 2026. https://github.com/openai/simple-evals/tree/652c89d0ca9df547706735883097e9537d40dc47

[9] Hugging Face. Math-Verify. Experiment uses version 0.9.0. Accessed 5 November 2026. https://github.com/huggingface/Math-Verify

[10] David Rein et al. GPQA dataset card and conditions concerning public disclosure of examples. Accessed 5 November 2026. https://huggingface.co/datasets/Idavidrein/gpqa

Annexure A Pre repair and post repair Sarvam execution

The following is a verbatim excerpt from the final frozen Sarvam worker, which contains both compared paths. For baseline, the full budget is sampled and the continuation branch is bypassed. For combined_repaired, the prefix budget is reduced by the reserve and the branch may insert a closing delimiter before continuing. No new after-the-fact implementation is substituted. Line numbers refer to the retained source; wrapped display continuations have no new number.

Listing A1 Actual baseline and repaired branches

Source work/sarvam30b-frozen-replication/worker.py lines 40 to 69

```
40      (R/'runtime.json').write_text(json.dumps(settings,indent=2))
41      for condition in ['baseline','combined_repaired']:
42          started=time.time();done=[];stage={}
43          assert not any((folder/(condition+'-'+b+'.jsonl')).exists() for b in
    ['humaneval_plus','math500','gpqa_diamond'])
44          for row in rows:
45              limit=row['output_budget']-(RESERVE if condition=='combined_repaired'
    else 0)
46              engine.add_request('prefix:'+row['id'],{'prompt_token_ids':prompts[con
    dition,row['id']]['ids']},sampling(limit))
47          while engine.has_unfinished_requests():
48              if time.time()>deadline:raise TimeoutError('Declared inference
    deadline reached')
49              for response in engine.step():
50                  if not response.finished:continue
51
    mode,k=response.request_id.split(':',1);sample=response.outputs[0];row=lookup[k]
52                  ids=list(sample.token_ids);end=next((i for i,x in enumerate(ids)
    if x in [1,26]),len(ids));ids=ids[:end]
53                  if condition=='combined_repaired' and mode=='prefix' and
    sample.finish_reason=='length':
54                      p=plan(tok,ids,row['output_budget']);forced=p['forced_ids'] if
     p else [];stage[k]=(ids,forced)
55                      remaining=row['output_budget']-len(ids)-len(forced);assert
    remaining>0
56                      engine.add_request('final:'+k,{'prompt_token_ids':prompts[cond
    ition,k]['ids']+ids+forced},sampling(remaining));continue
57                  prefix,forced=stage[k] if mode=='final' else ([],[])
58                  total=prefix+forced+ids;assert len(total)<=row['output_budget']
59
    raw=tok.decode(total,skip_special_tokens=True,clean_up_tokenization_spaces=False)
60                  final,method=final_channel(raw)
61                  record={'id':k,'task_id':k,'benchmark':row['benchmark'],'condition
    ':condition,'completion':raw,
62                      'final_completion':final,'final_channel_method':method,'comple
    tion_token_ids':total,
63                      'sampled_prefix_ids':prefix,'forced_token_ids':forced,'sampled
    _continuation_ids':ids,
64                      'new_tokens':len(total),'sampled_token_count':len(prefix)+len(
    ids),'policy_applied':bool(forced),
65                      'truncated':sample.finish_reason=='length','finish_reason':sam
    ple.finish_reason,'stop_reason':sample.stop_reason,
66                      'rendered_prompt_sha256':prompts[condition,k]['sha256'],'total
    _output_budget':row['output_budget']}
67                  with (folder/(condition+'-'+row['benchmark']+'.jsonl')).open('a')
    as f:f.write(json.dumps(record)+'\n')
68                  done.append(record)
69                  if len(done)%12==0:print('PROGRESS',condition,len(done),862,'secon
    ds',round(time.time()-started,1),flush=True)
```

The excerpt is context-dependent, not a stand-alone executable. The complete worker and supporting files are in annexure-sources/sarvam. Long lines are visually wrapped without changing the source file.

Annexure B Shared post repair structural policy

The core policy file is byte-identical between Sarvam and Qwen. Qwen supplies the context-aware tokenizer adapter shown later. The policy does not inspect a problem, an answer key or any correctness feedback.

Listing B1 Frozen policy and cutoff logic

Source work/qwen38-27b/budget_policy.py lines 1 to 20

```
"""Reserve final-answer space by ending an unfinished delimited reasoning span."""
import hashlib,json
RESERVE=1024

def open_reasoning(prefix_text):
    value=prefix_text.lstrip()
    return value.startswith('<think>') and '</think>' not in value

def plan(tokenizer,generated_ids,total_budget,reserve=RESERVE):
    assert total_budget>reserve>0
    cut=total_budget-reserve
    if len(generated_ids)<cut:return None
    prefix=list(generated_ids[:cut])
    text=tokenizer.decode(prefix,skip_special_tokens=True,clean_up_tokenization_sp
aces=False)
    if not open_reasoning(text):return None
    forced=tokenizer.encode('</think>\n',add_special_tokens=False)
    assert 0<len(forced)<reserve
    return
{'prefix_ids':prefix,'forced_ids':forced,'max_final_tokens':reserve-len(forced),
            'total_budget':total_budget,'prefix_sha256':hashlib.sha256(json.dumps(
prefix).encode()).hexdigest()}

```

The unchanged qualify function and full file are retained in the companion source package. The native tokenizer maps the same closing text to different token IDs for the two checkpoints.

Annexure C Pre repair and post repair Qwen execution

The Qwen unified worker also contains both arms. The exact excerpt below shows the full-budget baseline and repaired prefix/continuation scheduling. Splitting a sequence at the reserve is an operational change even when no delimiter is inserted; this is one reason to retain the trajectory and runtime limitations.

Listing C1 Actual Qwen sampler and compared branches

Source work/qwen38-27b/worker.py lines 61 to 89

```
61      params={}
62      def sampling(n):
63          if n not in params:params[n]=SamplingParams(temperature=0,top_p=1,max_toke
    ns=n,seed=settings['seed'],stop_token_ids=stop,detokenize=False)
64          return params[n]
65      def trim(ids):
66          ids=list(ids);end=next((i for i,x in enumerate(ids) if x in
    stop),len(ids));return ids[:end]
67      folder=R/'generations';folder.mkdir(exist_ok=True)
68      for c in ['baseline','combined_repaired']:
69          started=time.time();done=[]
70          assert not any((folder/(c+'-'+b+'.jsonl')).exists() for b in
    ['humaneval_plus','math500','gpqa_diamond'])
71          for offset in range(0,len(rows),48):
72              if time.time()>deadline:raise TimeoutError('Declared inference
    deadline reached')
73              batch=rows[offset:offset+48]
74              outputs=llm.generate([{'prompt_token_ids':prompts[c,r['id']]['ids']}
    for r in batch],
75                  [sampling(r['output_budget']-(1024 if c=='combined_repaired' else
    0)) for r in batch],use_tqdm=False)
76              saved={};continuations=[];continuation_params=[];continuation_ids=[]
77              for row,response in zip(batch,outputs):
78                  s=response.outputs[0];ids=trim(s.token_ids);p=prompts[c,row['id']]
79                  if c=='combined_repaired' and s.finish_reason=='length':
80                      plan=policy(tok,ids,row['output_budget'],p['opened_in_prompt']
    );extra=plan['forced_ids'] if plan else []
81                      saved[row['id']]=(ids,extra)
82                      continuations.append({'prompt_token_ids':p['ids']+ids+extra})
83
    continuation_params.append(sampling(row['output_budget']-len(ids)-len(extra)))
84                      continuation_ids.append(row['id'])
85                  else:saved[row['id']]=([],[],ids,s)
86              if continuations:
87
    tails=llm.generate(continuations,continuation_params,use_tqdm=False)
88                  for k,response in zip(continuation_ids,tails):
89                      prefix,extra=saved[k];s=response.outputs[0];saved[k]=(prefix,e
    xtra,trim(s.token_ids),s)
```

This excerpt preserves the frozen implementation. Native end markers are trimmed from retained output IDs. Full sampled and inserted token accounting and response recording appear in the complete worker.

Listing C2 Output accounting assertion

Source work/qwen38-27b/worker.py lines 90 to 93

```
90              for row in batch:
91                  prefix,extra,tail,s=saved[row['id']];ids=prefix+extra+tail
92                  assert len(ids)<=row['output_budget']
93                  p=prompts[c,row['id']];raw,logical,final,method=views(tok,ids,p['o
    pened_in_prompt'])
```

These lines assert the output ceiling and construct separate raw, logical and final views. Complete record serialization follows in the retained worker source.

Annexure D Native Qwen channel adapter

This model-interface adaptation is shared by the Qwen baseline and repaired arms. It reconstructs a marker already opened by the input template for logical parsing; it does not add that opening marker to the model's output-token budget.

Listing D1 Context-aware native token handling

Source work/qwen38-27b/adapter.py lines 1 to 22

```
  1  """Adapt unchanged structural policy to a template-opened native Qwen channel."""
  2  from budget_policy import plan
  3  from harness import final_channel
  4
  5  class ContextTokenizer:
  6      def __init__(self,tokenizer,opened_in_prompt):
  7          self.tokenizer=tokenizer;self.opened_in_prompt=opened_in_prompt
  8      def decode(self,ids,**kwargs):
  9          # Qwen marks thinking delimiters as special tokens; retain them literally.
 10          text=self.tokenizer.decode(ids,skip_special_tokens=False,clean_up_tokeniza
    tion_spaces=False)
 11          return ('<think>\n' if self.opened_in_prompt else '')+text
 12      def encode(self,text,**kwargs):
 13          return self.tokenizer.encode(text,add_special_tokens=False)
 14
 15  def policy(tokenizer,ids,budget,opened_in_prompt):
 16      return plan(ContextTokenizer(tokenizer,opened_in_prompt),ids,budget)
 17
 18  def views(tokenizer,ids,opened_in_prompt):
 19      sampled=tokenizer.decode(ids,skip_special_tokens=False,clean_up_tokenization_s
    paces=False)
 20      logical=('<think>\n' if opened_in_prompt else '')+sampled
 21      final,method=final_channel(logical)
 22      return sampled,logical,final,method
```

The remainder of adapter.py contains synthetic qualification fixtures. Full source is supplied; no benchmark item or reference key occurs in this adapter.

Annexure E Shared final channel parser and prompt audit

Listing E1 Final-channel extraction

Source work/qwen38-27b/harness.py lines 1 to 10

```
 1  """Reasoning-channel extraction independent of models, tasks and references."""
 2  def final_channel(text):
 3      stripped=text.lstrip()
 4      if not stripped.startswith('<think>'):return
    text,'no_leading_reasoning_channel'
 5      end=stripped.find('</think>',len('<think>'))
 6      if end<0:return '', 'incomplete_reasoning_no_final'
 7      final=stripped[end+len('</think>'):].lstrip()
 8      if final.startswith('<think>'):return '',
    'ambiguous_multiple_reasoning_channels'
 9      return final,'after_complete_reasoning_channel'
10
```

This shared parser is identical across both checkpoints and both arms. It is a view-extraction rule, not an independent source of correct answers.

The code and GPQA prompt pairs are byte-identical within each checkpoint: 164 of 164 and 198 of 198 rendered prompt hashes match. All 500 mathematics pairs differ by the same declared generic instruction change, with the original question preserved. The repaired general instruction is: Solve the following math problem step by step. End your response with a line that starts with Answer: followed by your final result. The baseline uses the pinned simple-evals answer-placeholder format and final reminder [8].

There are no benchmark-specific answer branches in the structural policy. Worker routing identifiers and per-task output budgets are orchestration metadata, not solution hints. This source audit does not exclude benchmark-informed development or training-data contamination. The companion package intentionally omits the full question projection because it contains GPQA examples.

Annexure F Reproducibility identifiers

SHA-256 values below refer to immutable retained artifacts. The companion manifest contains full hashes, source paths and public-copy provenance. Public protocol copies omit personal funding fields and therefore have their own distinct hashes; the original frozen protocol hashes remain listed for identity.

Sarvam protocol
`b382530fbd7149e8fded6a5316dd2855`
`f19eff505797fb957351c84ebdb42ee3`

Qwen protocol
`43b2403ae1b331ad485df8c43d115852`
`d27331ffea6d6dad9fa3df245991e6af`

Sarvam worker
`a2e652eb4d88c2439931cfede6f93a67`
`46eb3c3f26efba89d266b61b407126f3`

Qwen worker
`14507bcd1b5d380dd86c9313f8c1bcd8`
`462b5433b852ca31a8d4c7dc7759d0b7`

Shared budget policy
`092dacddb421e769b86829a24f387976`
`fb782602dfda4a334efe7d7c24169ff5`

Shared channel parser
`1d3091c1c9f76e820efc2c4c84c010ac`
`2d935ebee21170c375e39d5aa6cc3cc3`

Qwen adapter
`ecf32416beb1e84a21b147a0ed66b98c`
`49124f3b96a96bc92bfde0b939c09d53`

Table F1 Dataset and evaluation identities

Artifact	Identifier
HumanEvalPlus	v0.1.10; dataset MD5 fe585eb4df8c88d844eeb463ea4d0302
MATH500 CSV SHA-256	830003254c2894832217e1da6ed7fd93 04546d2161640c1ce05ab9cccf532e2f
GPQA Diamond CSV SHA-256	41d1213cd7a4998605a26c2798500652 572007161b3a92817ba46b35befcd305
simple-evals protocol commit	652c89d0ca9df547706735883097e9537d40dc47
Shared question-only projection SHA-256	732f96bca7bafc621e65092e3ea343a6 b20fd3cf817308aefa1a5e36a6039bba

The final Qwen choice-freeze ledger hash is 4c3242a2a2e91aabcd0784724338f18c0beba66ab7dd2dd111d6738af83b9216; the Sarvam choice-freeze hash is 28eb13093c59732fa6f5b9ff576c7865b1099f3447ef7ddbd4b1e4bd19d167d9. The ledgers themselves remain private because they contain selected choices. Hashes commit to artifact identity and the recorded freeze ordering; they do not independently certify scientific truth.